CADRIA

Copilote de cadrage et pilotage de projets

Strategie Git & Travail d'equipe

Equipe de 3 personnes - Trace Git pour l'evaluation

Base de depart : commit initial de Houssem

Repo : github.com/houssem4512/CADRIA.git

Contenu de base : backend (Django) + frontend (React) + docs + docker

Branche de reference : main (commit Initial commit ce182cb)

Objectif : chacun construit sa partie SANS ecraser la base.

Date : 14 octobre 2026 - Document a partager dans le groupe

Important : ce document est la regle commune. Tout le monde l'applique des aujourd'hui.

1. Ce que le prof va verifier

Le prof ne veut pas un ZIP final. Il veut la TRACE Git : qui a fait quoi, quand, et comment. Il regardera :

- git log : liste des commits avec auteur + date + message
- git shortlog -sne : nombre de commits par personne (il faut 3 noms visibles)
- git blame : auteur ligne par ligne dans chaque fichier
- Sur GitHub : Insights > Contributors / Commits / Network (graphique d'activite)

S'il ne voit qu'1 seul commit 'Initial commit' ou 1 seul auteur, c'est considere comme travail non reparti, meme si le code est bon.

2. Point de depart : la base de Houssem

Aujourd'hui, main contient la base complete : backend Django + frontend React + docs + docker-compose. C'est le socle. On ne le recommence pas, on ne le supprime pas.

- main = branche stable et protegee. On ne code jamais directement dedans.
- Chaque membre cree sa branche a partir de main, travaille dedans, puis fusionne via Pull Request.
- Resultat attendu : main avance par fusions successives, avec historique propre.

```
main (stable)
  |-- feat/frontend-dashboard-dali
  |-- feat/backend-taches-houssem
  |-- feat/agents-risques-membre3
  |-- docs/architecture
  |-- fix/bug-auth
```

3. Les 3 roles - notez vos noms

Notez vos noms ici. Chacun est responsable de son perimetre, mais tout le monde relit les Pull Requests des autres.

Membre A : Frontend (React + Tailwind)

- Dossier : frontend/src/pages, components, context, lib
- Branches : feat/front-...
- Exemples : dashboard KPI, kanban, pages admin, toasts, charts

Membre B : Backend (Django + API)

- Dossier : backend/accounts, projects, notifications, analytics
- Branches : feat/back-...
- Exemples : endpoints tasks/sprints, auth JWT, permissions, tests

Membre C : Agents + Pilotage + Docs / DevOps

- Dossier : backend/orchestration, audit, docs, docker-compose
- Branches : feat/agents-... , docs/...
- Exemples : services.py dispatch(run), payloads, journal audit, docs API/AGENTS/ARCHITECTURE

Houssem (proprietaire du repo) garde le role de referent main : il valide et fusionne les Pull Requests. Si c'est un autre, designez 1 referent.

4. Installation initiale - a faire une fois par chacun

Etape 1 - Le proprietaire invite les 2 autres : sur GitHub > CADRIA > Settings > Collaborators > Add people (avec leurs comptes GitHub).

Etape 2 - Sur SON PC, chacun configure SON identite (obligatoire pour la trace) :

```
git config --global user.name "Prenom NOM"
git config --global user.email "mon-email-github@example.com"
git config user.name  # verifier
git config user.email # verifier
```

Etape 3 - Chacun clone puis prepare main :

```
git clone https://github.com/houssem4512/CADRIA.git
cd CADRIA
git checkout main
git pull origin main
```

Erreur a ne pas faire : travailler tous sur le meme PC / meme compte GitHub. La trace serait faussee.

5. Workflow quotidien - le cycle obligatoire

Toujours le meme cycle : sync > branche > petits commits > push > Pull Request > merge. Jamais de code envoye en ZIP / WhatsApp.

```
# 1. Se mettre a jour
git checkout main
git pull origin main

# 2. Creer / reprendre SA branche
git checkout -b feat/front-kanban-dali
# ou : git checkout feat/front-kanban-dali
```

```
# 3. Travailler + commiter SOUVENT
git status
git add frontend/src/pages/Kanban.jsx
git commit -m "feat(front): grille kanban"

# 4. Pousser
git push -u origin feat/front-kanban-dali
```

Ensuite sur GitHub : Pull Request vers main > 1 collegue relit > Merge > supprimer la branche distante > git pull sur main en local. Objectif : 1 a 2 commits par personne et par jour de travail.

6. Convention de commits - messages propres

Format : type(perimetre): description courte. Exemples a copier :

```
feat(front): ajout KPI velocite dashboard
feat(back): endpoint bulk_status taches
feat(agents): payload scoping + gestion SKIPPED
fix(auth): refresh JWT expire
docs: MAJ ARCHITECTURE + schema BDD
test: 3 tests kanban move
```

Types autorises : feat, fix, docs, style, test, chore. Interdits : ok, final, modif, projet fini. Un commit = un changement logique, pas 20 fichiers melanges.

Avant chaque commit, verifiez : git status + git diff --stat. N'ajoutez jamais .venv, node_modules, .env (deja dans .gitignore).

7. Regles d'or et interdits

- INTERDIT : 1 seul gros commit final le dernier jour. C'est zero pour la trace.
- INTERDIT : commit avec le compte d'un autre. Chacun son PC + son compte.
- INTERDIT : force push sur main (git push --force). Demandez au referent en cas de conflit.
- OBLIGATOIRE : pull de main avant de commencer chaque jour pour eviter les conflits.
- OBLIGATOIRE : Pull Request meme pour une petite modif, avec titre clair + description.
- En cas de conflit : git pull origin main dans votre branche, resolvez, testez, puis commit.

8. Preuve finale pour le prof - le jour J

Le jour de la remise, le referent genere la preuve en 2 minutes :

```
git checkout main
git pull origin main
git log --graph --oneline --all --decorate
git shortlog -sne --all
git log --pretty=format:"%h - %an - %ad - %s" --date=short --all > TRACE_GIT.txt
```

A joindre au rapport :

- 1. Capture GitHub Insights > Contributors (on doit voir 3 noms + courbes)
- 2. Capture Network (branches fusionnees dans main)
- 3. Fichier TRACE_GIT.txt + git blame sur 2 fichiers cles
- 4. Liens des Pull Requests fusionnees (preuve de relecture)

Verif rapide : si git shortlog -sne --all n'affiche pas 3 lignes avec vos 3 emails, il y a un probleme d'identite a

corriger immediatement.

9. Checklist d'engagement - a signer

- [ ] Les 3 identites git config sont faites et verifiees
- [ ] Les 2 collaborateurs sont invites sur GitHub
- [ ] Les 3 roles sont attribues (noms notes ci-dessus)
- [ ] Chacun a pousse au moins 1 branche + 1 Pull Request cette semaine
- [ ] main reste verte : tests backend OK + frontend build OK avant chaque merge

Bon courage. La base de Houssem est solide : a vous de construire dessus, chacun sa partie, avec une trace propre. Le prof doit pouvoir raconter votre projet rien qu'en lisant l'historique Git.